

MSYNC: pseudosynchrony

I. S. Plank

2025-07-15

Introduction

This R Markdown script compares pseudosynchrony to synchrony values for the MSYNC project. The data was preprocessed before being read into this script. We use two approaches described in Moulder et al. (2018, Psychol Methods) to test the following hypotheses: “Synchrony does not exist between sections of size m in these two time series” (section sliding) and, if the first hypothesis is rejected, “There is no time dependency at all between these two time series” (data shuffling). We use the same settings as when computing synchrony to measure synchrony (windowed-lagged cross-correlation with the same timing settings).

Package versions

The following packages are used in this RMarkdown file:

```
## [1] "R version 4.5.1 (2025-06-13)"
## [1] "knitr version 1.50"
## [1] "tidyverse version 2.0.0"
## [1] "ggpubr version 0.6.0"
## [1] "BayesFactor version 0.9.12.4.7"
## [1] "effectsize version 1.0.1"
## [1] "gggrain version 0.0.4"
## [1] "rMEA version 1.2.2"
```

MEA data

```
if (!file.exists(file.path(dt.path, "preprocessedMEA", "df_psync.csv"))) {

  # create a dataframe
  df.psync = data.frame()

  # check if CT has been run before
  if (!file.exists(file.path(dt.path, "preprocessedMEA", "df_psync-CT.csv"))) {
    # start with the conversation task
    attach(file.path(dt.path, "preprocessedMEA", "MEA_CT.Rdata"), name = 'CT')
    for (meth in c("seg", "data")) {
      df.meas = pseudosync(meth, meas.scaled,
                           sampRate = 120,
                           lagSec = 5,
                           winSec = 30,
                           incSec = 15,
                           n = n)

      df.meas = df.meas %>%
        mutate(
          dyad = sprintf("MSYNC_%s", gsub("all_(.)_0.*", "\\1", name)),
          task = "CT",
          method = meth
        ) %>%
        merge(., df.ccf.agg %>% ungroup() %>%
              filter(position == "B" & measure == "mean") %>%
              select(dyad, phase, MEA.sync))
    }
  }
}
```

```

df.psync = rbind(df.psync, df.mea)
write_csv(df.psync, file.path(dt.path, "preprocessedMEA", "df_psync-CT.csv"))
}
# detach the CT data
detach('CT')
} else {
df.psync = read_csv(file.path(dt.path, "preprocessedMEA", "df_psync-CT.csv"))
}

# # check if CT has been run before
# if (!file.exists(file.path(dt.path, "preprocessedMEA", "df_psync-MG.csv"))) {
#   # at the mirror game
#   attach(file.path(dt.path, "preprocessedMEA", "MEA_MG.Rdata"), name = 'MG')
#   for (meth in c("seg", "data")) {
#     df.mea = pseudosync(meth, mea.scaled,
#                         sampRate = 120,
#                         lagSec = 2,
#                         winSec = 30,
#                         incSec = 1,
#                         n = n)
#     df.mea = df.mea %>%
#       mutate(
#         dyad = sprintf("MSYNC_%s", gsub("all_(.+)_0.*", "\\1", name)),
#         task = "MG",
#         method = meth,
#         phase = as.numeric(substr(name, nchar(name), nchar(name)))
#       ) %>%
#     merge(., df.ccf.agg %>% ungroup() %>%
#       filter(position == "B" & measure == "mean") %>%
#       select(dyad, phase, MEA.sync))
#     df.psync = rbind(df.psync, df.mea)
#     write_csv(df.psync, file.path(dt.path, "preprocessedMEA", "df_psync-MG.csv"))
#   }
#   # detach the MG data
#   detach('MG')
# } else {
#   df.mea = read_csv(file.path(dt.path, "preprocessedMEA", "df_psync-MG.csv"))
#   df.psync = rbind(df.psync, df.mea)
# }

# save the pseudosynchrony values
write_csv(df.psync, file.path(dt.path, "preprocessedMEA", "df_psync.csv"))
} else {
df.psync = read_csv(file.path(dt.path, "preprocessedMEA", "df_psync.csv"))
}

```

```

df.pagg = df.psync %>%
group_by(dyad, task, method) %>%
summarise(
  psync = mean(as.numeric(psync)),
  MEA.sync = mean(MEA.sync)
) %>%
pivot_longer(cols = c(psync, MEA.sync)) %>%
mutate_if(is.character, as.factor)

```

```

aov.CT.seg = anovaBF( value ~ name,
                     df.pagg %>% filter(task == "CT" & method == "seg"))

```

```

aov.CT.seg@bayesFactor

```

```
##          bf          error          time          code
## name -0.9064766 0.0001514081 Tue Jul 15 18:40:10 2025 6ef52b9d5776
```

```
interpret_bf(aov.CT.seg@bayesFactor$bf, log = T)
```

```
## [1] "anecdotal evidence against"
```

```
## (Rules: jeffreys1961)
```

```
aov.CT.data = anovaBF( value ~ name,
                      df.pagg %>% filter(task == "CT" & method == "data"))
```

```
aov.CT.data@bayesFactor
```

```
##          bf          error          time          code
```

```
## name 72.11793 1.599553e-37 Tue Jul 15 18:40:10 2025 6ef566d91a58
```

```
interpret_bf(aov.CT.data@bayesFactor$bf, log = T)
```

```
## [1] "extreme evidence in favour of"
```

```
## (Rules: jeffreys1961)
```

```
p1a = df.pagg %>%
  filter(task == "CT" & method == "seg") %>%
  ggplot(., aes(x = name, y = value, fill = name, colour = name)) + #
  geom_rain(rain.side = 'f1x1',
            boxplot.args = list(color = "black", outlier.shape = NA, show_guide = FALSE, alpha = a),
            violin.args = list(color = "black", outlier.shape = NA, alpha = a),
            point.args = list(show_guide = FALSE, alpha = a)
          ) +
  stat_summary(fun = mean, geom = "line", aes(group = 1), colour = "black") +
  stat_summary(fun = mean, geom = "point", shape = 23, size = 2,
              aes(group = name)) +
  stat_summary(fun = mean, geom = "point", shape = 5, size = 2,
              aes(group = name), colour = "black") +
  scale_fill_manual(values = custom.col[c(1,3)]) +
  scale_colour_manual(values = custom.col[c(1,3)]) +
  labs(title = "Conversation task: segments",
       x = "",
       y = "") +
  theme_bw() +
  theme(legend.position = "none",
        plot.title = element_text(hjust = 0.5),
        text = element_text(size = 12))
```

```
p1b = df.pagg %>%
  filter(task == "CT" & method == "data") %>%
  ggplot(., aes(x = name, y = value, fill = name, colour = name)) + #
  geom_rain(rain.side = 'f1x1',
            boxplot.args = list(color = "black", outlier.shape = NA, show_guide = FALSE, alpha = a),
            violin.args = list(color = "black", outlier.shape = NA, alpha = a),
            point.args = list(show_guide = FALSE, alpha = a)
          ) +
  stat_summary(fun = mean, geom = "line", aes(group = 1), colour = "black") +
  stat_summary(fun = mean, geom = "point", shape = 23, size = 2,
              aes(group = name)) +
  stat_summary(fun = mean, geom = "point", shape = 5, size = 2,
              aes(group = name), colour = "black") +
  scale_fill_manual(values = custom.col[c(1,3)]) +
  scale_colour_manual(values = custom.col[c(1,3)]) +
  labs(title = "Conversation task: data shuffling",
       x = "",
       y = "") +
  theme_bw() +
  theme(legend.position = "none",
```

```

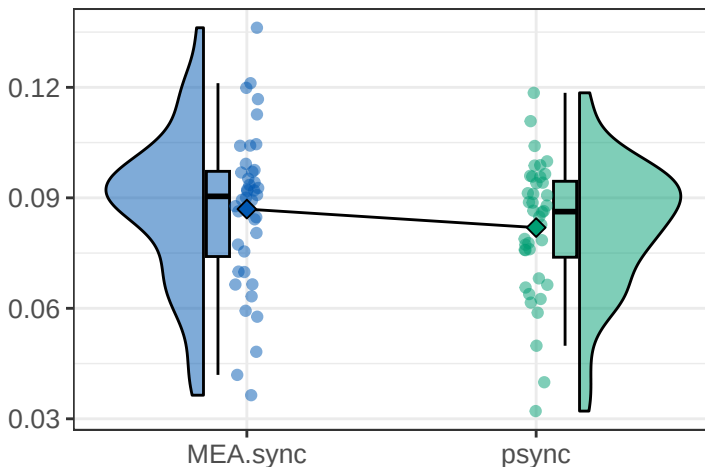
plot.title = element_text(hjust = 0.5),
text = element_text(size = 12))

# aov.MG = anovaBF( value ~ name,
#                  df.pagg %>% filter(task != "CT"))
#
# aov.MG@bayesFactor
# interpret_bf(sort(aov.MG@bayesFactor$bf, decreasing = T)[1], log = T)
#
# p2 = df.pagg %>%
#   filter(task != "CT") %>%
#   ggplot(., aes(x = name, y = value, fill = name, colour = name)) + #
#   geom_rain(rain.side = 'f1x1',
#             boxplot.args = list(color = "black", outlier.shape = NA, show_guide = FALSE, alpha = a),
#             violin.args = list(color = "black", outlier.shape = NA, alpha = a),
#             point.args = list(show_guide = FALSE, alpha = a)
#           ) +
#   stat_summary(fun = mean, geom = "line", aes(group = 1), colour = "black") +
#   stat_summary(fun = mean, geom = "point", shape = 23, size = 2,
#               aes(group = name)) +
#   stat_summary(fun = mean, geom = "point", shape = 5, size = 2,
#               aes(group = name), colour = "black") +
#   scale_fill_manual(values = custom.col[c(1,3)]) +
#   scale_colour_manual(values = custom.col[c(1,3)]) +
#   labs(title = "Mirror game",
#        x = "",
#        y = "") +
#   theme_bw() +
#   theme(legend.position = "none",
#         plot.title = element_text(hjust = 0.5),
#         text = element_text(size = 12))

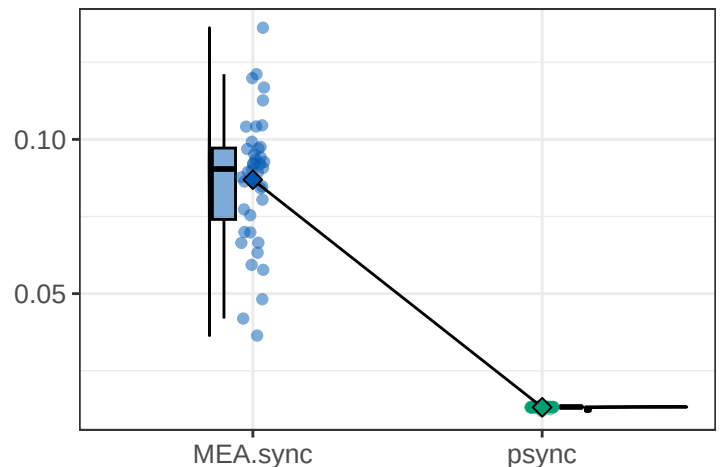
ggarrange(p1a, p1b, #p2,
          ncol = 2)

```

Conversation task: segments



Conversation task: data shuffling



[!ADD]

Let's see whether we have at least some dyads that exceeded pseudosynchrony in the conversation task.

```

# initialise a dataframe
df.sub = data.frame()

# loop through dyads
for (d in unique(df.psync$dyad)) {
  df.sub = df.psync %>% filter(method == "seg" & dyad == d)
}

```

```

t = ttestBF(x = df.seg$psync, mu = mean(df.seg$MEA.sync))
df.sub = rbind(df.sub,
  data.frame(
    dyad = d,
    bf.log = t@bayesFactor$bf,
    psync = mean(df.seg$psync),
    MEA.sync = mean(df.seg$MEA.sync)
  ))
}

df.sub = df.sub %>%
  mutate(
    direction = if_else(psync < MEA.sync,
      "less", "greater"),
    relevant = if_else(direction == "less" & bf.log >= 3, T, F),
    interpretation = interpret_bf(t@bayesFactor$bf, log = T)
  ) %>% write_csv(., file.path(dt.path, "preprocessedMEA", "df_psync_sub.csv"))

```

About half the dyads did not exceed pseudosynchrony when shuffling the segments. We might want to come back and investigate this later.

Speech data

Individual level: turn-based pseudosynchronisation

For turn-based pseudosynchronisation, this script uses an approach described in Moulder et al. (2018, *Psychol Methods*) to test the following hypothesis: “Synchrony does not exist between sections of size m in these two time series” (section sliding). This segment shuffling, from Moulder et al., “requires researchers to cut a time series X into shorter sections of size m which are randomly appended to one another to create a new time series X_s until no section is in its original position”. This new time series is then paired with an unchanged time series Y . Since the synchronisation is already based on sections or speech turns in the conversation, no data shuffling will be performed. Since each dyad can have a unique amount of turns, no dyad shuffling will be performed.

For each speaker in each task, the order of the previous turns are shuffled 100 times while the order of the current turns remain unchanged. Note that no check for uniqueness is performed. In the end, an average for this specific speaker and task is calculated from the 100 pseudosynchrony values.

```

filename = "indi_st-shuffle.RData"

if (!file.exists(filename)) {

  speaker = c()
  psyncint = c()
  psyncpit = c()
  psyncart = c()

  df = read_csv(file.path(dt.path, "preprocessedAudio", "OUT_turns.csv"),
    show_col_types = F) %>%
    mutate(speaker = paste(name, speaker, sep = "_")) %>%
    select(c(speaker, pit_turn, pit_prev_turn,
      int_turn, int_prev_turn,
      art_prev_turn, art_turn))

  for (s in unique(df$speaker)) {
    speaker = c(speaker, rep(s, times = n))
    df.sel = df %>% filter(speaker == s)
    for (i in 1:n) {
      # shuffling
      df.sel$pit_prev = sample(df.sel$pit_prev_turn)
      df.sel$int_prev = sample(df.sel$int_prev_turn)
      df.sel$art_prev = sample(df.sel$art_prev_turn)
      psyncint = c(psyncint, abs(cor(df.sel$int_turn, df.sel$int_prev, use = "na.or.complete")))
    }
  }
}

```

```

    psyncpit = c(psyncpit, abs(cor(df.sel$pit_turn,df.sel$pit_prev, use = "na.or.complete")))
    psyncart = c(psyncart, abs(cor(df.sel$art_turn,df.sel$art_prev, use = "na.or.complete")))
  }
}
df.s_shuffle = data.frame(speaker,psyncint, psyncpit, psyncart)
save(df.s_shuffle,file = file.path(dt.path, "preprocessedAudio", filename))
} else {
  load(filename)
}

```

```

df.sync = read_csv(file.path(dt.path, "preprocessedAudio", "OUT_sync-indi.csv"),
  show_col_types = F) %>%
  mutate(
    speaker = paste(name, speaker, sep = "_")
  ) %>%
  select(c(speaker, pit_sync, int_sync, art_sync))

```

```

df.agg = df.s_shuffle %>% group_by(speaker) %>%
  summarise(
    pit_psync = mean(psyncpit),
    int_psync = mean(psyncint),
    art_psync = mean(psyncart)
  )

```

```
df.agg = merge(df.sync,df.agg)
```

```

res = ttestBF(x = df.agg[!is.na(df.agg$pit_sync),]$pit_sync,
  y = df.agg[!is.na(df.agg$pit_sync),]$pit_psync,
  paired = T)

```

```
res
```

```

## Bayes factor analysis
## -----
## [1] Alt., r=0.707 : 5.789085 ±0%
##
## Against denominator:
##   Null, mu = 0
## ---
## Bayes factor type: BFoneSample, JZS

```

```
res@bayesFactor[["bf"]]
```

```
## [1] 1.755974
```

```

res = ttestBF(x = df.agg[!is.na(df.agg$int_sync),]$int_sync,
  y = df.agg[!is.na(df.agg$int_sync),]$int_psync,
  paired = T)

```

```
res
```

```

## Bayes factor analysis
## -----
## [1] Alt., r=0.707 : 1953.851 ±0%
##
## Against denominator:
##   Null, mu = 0
## ---
## Bayes factor type: BFoneSample, JZS

```

```
res@bayesFactor[["bf"]]
```

```
## [1] 7.577558
```

```

res = ttestBF(x = df.agg[!is.na(df.agg$art_sync),]$art_sync,
  y = df.agg[!is.na(df.agg$art_sync),]$art_psync,

```

```

        paired = T)
res

## Bayes factor analysis
## -----
## [1] Alt., r=0.707 : 157.8331 ±0%
##
## Against denominator:
##   Null, mu = 0
## ---
## Bayes factor type: BFoneSample, JZS
res@bayesFactor[["bf"]]

## [1] 5.061538

```

The t-tests show that there is a significant difference between the pseudosynchronisation values obtained through segment shuffling and the real synchronisation values in the case of turn-based synchronisation of intensity, pitch and articulation rate, with the real synchronisation values being higher.

Dyad level: timecourse pseudosynchrony based on CCF

We again use segment shuffling to compare pseudosynchrony and synchrony values on the dyad level. Pitch and intensity synchrony was calculated using the CCF function from rMEA. Therefore, we use the same function with the same window size but with shuffled windows for calculating pseudosynchrony.

For each speaker in each task, their time series is shuffled 50 times and then paired with the unchanged time series of their partner in the dyad. Note that no check for uniqueness is performed. In the end, an average for this specific dyad and task is calculated from the 100 pseudosynchrony values.

```

filename = "dyad_s-shuffle.RData"

if (!file.exists(file.path(dt.path, "preprocessedAudio", filename))) {

  # create new data frame for pseudosync values
  cols = c("dyad", "task", "pit_psync_MEA", "int_psync_MEA")
  df.psync = data.frame(matrix(nrow = 0, ncol = length(cols)))
  colnames(df.psync) = cols

  # load actual sync values
  df.dyad = read_csv(file.path(dt.path, "preprocessedAudio", "OUT_dyad.csv"),
                     show_col_types = F)

  sr = 100

  # go through all dyads
  for (i in 1:nrow(df.dyad)) {
    # load the respective data of continuous pitch and intensity
    ptn = paste(df.dyad$dyad[i], df.dyad$task[i], ".*", "cont.csv", sep = "_")
    fls = list.files(pattern = ptn, path = file.path(dt.path, "preprocessedAudio"))
    df_L = read_csv(file.path(dt.path, "preprocessedAudio", fls[1]),
                    show_col_types = F, col_types = cols(.default = "d"))
    df_R = read_csv(file.path(dt.path, "preprocessedAudio", fls[2]),
                    show_col_types = F, col_types = cols(.default = "d"))

    # resample to 100Hz
    L_pit = df_L$pitch
    L_pit[is.na(L_pit)] = 0
    L_pit = signal::resample(L_pit, 1, 1000/sr)
    L_int = df_L$int
    L_int[is.na(L_int)] = 0
    L_int = signal::resample(L_int, 1, 1000/sr)
    R_pit = df_R$pitch
    R_pit[is.na(R_pit)] = 0
  }
}

```

```

R_pit = signal::resample(R_pit,1,1000/sr)
R_int = df_R$int
R_int[is.na(R_int)] = 0
R_int = signal::resample(R_int,1,1000/sr)
# create fakeMEA and divide into segments
mea_int = fakeMEA(L_int,R_int,sr,"int_L","int_R")
mea_pit = fakeMEA(L_pit,R_pit,sr,"pit_L","pit_R")
mea_int_L = split(mea_int[["all_01_01"]][["MEA"]][["s1Name"]], floor(seq_along(mea_int[["all_01_01"]][["MEA"]]) / 2))
mea_int_R = split(mea_int[["all_01_01"]][["MEA"]][["s2Name"]], floor(seq_along(mea_int[["all_01_01"]][["MEA"]]) / 2))
mea_pit_L = split(mea_pit[["all_01_01"]][["MEA"]][["s1Name"]], floor(seq_along(mea_pit[["all_01_01"]][["MEA"]]) / 2))
mea_pit_R = split(mea_pit[["all_01_01"]][["MEA"]][["s2Name"]], floor(seq_along(mea_pit[["all_01_01"]][["MEA"]]) / 2))
# shuffle each side n/2 times
for (j in 1:(n/2)) {

  # shuffling left
  mea_int[["all_01_01"]][["MEA"]][,2] = unlist(mea_int_R)
  mea_int[["all_01_01"]][["MEA"]][,1] = unlist(sample(mea_int_L))
  mea_int = MEAccf(mea_int,2,16,8)
  mea_pit[["all_01_01"]][["MEA"]][,2] = unlist(mea_pit_R)
  mea_pit[["all_01_01"]][["MEA"]][,1] = unlist(sample(mea_pit_L))
  mea_pit = MEAccf(mea_pit,2,16,8)
  # create new row with mean of peaks of windows
  new = c(df.dyad$dyad[i],
          df.dyad$task[i],
          mean(apply(mea_int$all_01_01$ccf,1,max)),
          mean(apply(mea_pit$all_01_01$ccf,1,max)))
  df.psync[nrow(df.psync) + 1,] = new

  # shuffling right
  mea_int[["all_01_01"]][["MEA"]][,2] = unlist(sample(mea_int_R))
  mea_int[["all_01_01"]][["MEA"]][,1] = unlist(mea_int_L)
  mea_int = MEAccf(mea_int,2,16,8)
  mea_pit[["all_01_01"]][["MEA"]][,2] = unlist(sample(mea_pit_R))
  mea_pit[["all_01_01"]][["MEA"]][,1] = unlist(mea_pit_L)
  mea_pit = MEAccf(mea_pit,2,16,8)
  # create new row with mean of peaks of windows
  new = c(df.dyad$dyad[i],
          df.dyad$task[i],
          mean(apply(mea_int$all_01_01$ccf,1,max)),
          mean(apply(mea_pit$all_01_01$ccf,1,max)))
  df.psync[nrow(df.psync) + 1,] = new
}
}
# save everything
save(df.psync, file = file.path(dt.path, "preprocessedAudio", filename))
} else {
  load(file.path(dt.path, "preprocessedAudio", filename))
}

```

```

df.sync = read_csv(file.path(dt.path, "preprocessedAudio", "OUT_dyad.csv"),
                   show_col_types = F) %>%
  select(c(dyad, task, pit_sync_MEA, int_sync_MEA))

df.agg = df.psync %>%
  mutate(
    pit_psync_MEA = as.numeric(pit_psync_MEA),
    int_psync_MEA = as.numeric(int_psync_MEA)
  ) %>%
  group_by(dyad, task) %>%
  summarise(
    pit_psync_MEA = mean(pit_psync_MEA),

```

```

    int_psync_MEA = mean(int_psync_MEA)
  )

df.agg = merge(df.sync, df.agg)

res = ttestBF(x = df.agg[!is.na(df.agg$pit_sync_MEA),]$pit_sync_MEA,
             y = df.agg[!is.na(df.agg$pit_sync_MEA),]$pit_psync_MEA,
             paired = T)
res

```

```

## Bayes factor analysis
## -----
## [1] Alt., r=0.707 : 41.52436 ±0%
##
## Against denominator:
##   Null, mu = 0
## ---
## Bayes factor type: BFoneSample, JZS
res@bayesFactor[["bf"]]

```

```

## [1] 3.72628
res = ttestBF(x = df.agg[!is.na(df.agg$int_sync_MEA),]$int_sync_MEA,
             y = df.agg[!is.na(df.agg$int_sync_MEA),]$int_psync_MEA,
             paired = T)
res

```

```

## Bayes factor analysis
## -----
## [1] Alt., r=0.707 : 2.496086e+22 ±0%
##
## Against denominator:
##   Null, mu = 0
## ---
## Bayes factor type: BFoneSample, JZS
res@bayesFactor[["bf"]]

```

```
## [1] 51.5716
```

The t-tests show that there is a significant difference between the pseudosynchrony values obtained through segment shuffling and the real synchrony values in the case of MEA-based CCF synchrony of intensity and pitch, with the real synchrony values being higher.

OpenPose data

```

if (!file.exists(file.path(dt.path, "preprocessedOP", "df_psync.csv"))) {

  # create a dataframe
  df.psync = data.frame()

  for (task in c("CT", "MG")) {
    # get the data
    attach(file.path(dt.path, "preprocessedOP",
                     sprintf("OpenFace_%s.Rdata", task)), name = task)
    for (meth in c("seg", "data")) {
      df.mea = pseudosync(meth, ls.fakeAU,
                         sampRate = 120,
                         lagSec = 2, winSec = 7, incSec = 4,
                         n = n)

      df.mea = df.mea %>%
        mutate(

```

```

    dyad = gsub(sprintf("(.)_%s.*", task), "\\1", name),
    phase = task,
    method = meth,
    key = gsub(sprintf(".*_%s_(.)", task), "\\1", name),
    psync = as.numeric(psync)
  ) %>%
  merge(., readRDS(file.path(dt.path,
                             sprintf("MSYNC_AU_sync_%s.rds")))) %>%
    ungroup() %>%
    filter(position == "B" & measure == "mean") %>%
    select(dyad, phase, key, OF.sync))
df.psync = rbind(df.psync, df.meas)
}
# detach the data
detach(task)

}

# save the pseudosynchrony values
write_csv(df.psync, file.path(dt.path, "preprocessedOF", "df_psync.csv"))
} else {
  df.psync = read_csv(file.path(dt.path, "preprocessedOF", "df_psync.csv"))
}

```

OpenFace data

```

if (!file.exists(file.path(dt.path, "preprocessedOF", "df_psync.csv"))) {

  # create a dataframe
  df.psync = data.frame()

  # only do the conversation task
  attach(file.path(dt.path, "preprocessedOF", "OpenFace_CT.Rdata"), name = 'CT')
  for (meth in c("seg", "data")) {
    df.meas = pseudosync(meth,
                          ls.fakeAU,
                          sampRate = 120,
                          lagSec = 2, winSec = 7, incSec = 4,
                          n = n)

    df.meas = df.meas %>%
      mutate(
        dyad = gsub("(.)_CT.*", "\\1", name),
        phase = "CT",
        method = meth,
        key = gsub(".*_CT_(.)", "\\1", name),
        psync = as.numeric(psync)
      ) %>%
      merge(., readRDS(file.path(dt.path, "MSYNC_AU_sync_CT.rds"))) %>%
        ungroup() %>%
        filter(position == "B" & measure == "mean") %>%
        select(dyad, phase, key, OF.sync))
    df.psync = rbind(df.psync, df.meas)
  }
  df.psync = rbind(df.psync, df.meas)
  # detach the CT data
  detach('CT')

  # save the pseudosynchrony values

```

```

write_csv(df.psync, file.path(dt.path, "preprocessedOF", "df_psync.csv"))
} else {
  df.psync = read_csv(file.path(dt.path, "preprocessedOF", "df_psync.csv"))
}

df.pagg = df.psync %>%
  mutate(
    type = if_else(substr(key, 1, 2) == "AU", "AU", "pose")
  ) %>%
  group_by(dyad, type) %>%
  summarise(
    psync = mean(psync),
    OF.sync = mean(OF.sync)
  ) %>%
  pivot_longer(cols = c(psync, OF.sync)) %>%
  mutate_if(is.character, as.factor)

aov.CT.AU = anovaBF( value ~ name ,
                    df.pagg %>% filter(type == "AU"))

aov.CT.AU@bayesFactor
interpret_bf(aov.CT.AU@bayesFactor$bf, log = T)

aov.CT.PS = anovaBF( value ~ name ,
                    df.pagg %>% filter(type == "pose"))

aov.CT.PS@bayesFactor
interpret_bf(aov.CT.PS@bayesFactor$bf, log = T)

p1 = df.pagg %>%
  filter(task == "CT") %>%
  ggplot(., aes(x = name, y = value, fill = name, colour = name)) + #
  geom_rain(rain.side = 'f1x1',
            boxplot.args = list(color = "black", outlier.shape = NA, show_guide = FALSE, alpha = a),
            violin.args = list(color = "black", outlier.shape = NA, alpha = a),
            point.args = list(show_guide = FALSE, alpha = a)
          ) +
  stat_summary(fun = mean, geom = "line", aes(group = 1), colour = "black") +
  stat_summary(fun = mean, geom = "point", shape = 23, size = 2,
              aes(group = name)) +
  stat_summary(fun = mean, geom = "point", shape = 5, size = 2,
              aes(group = name), colour = "black") +
  scale_fill_manual(values = custom.col[c(1,3)]) +
  scale_colour_manual(values = custom.col[c(1,3)]) +
  labs(title = "Conversation task",
       x = "",
       y = "") +
  theme_bw() +
  theme(legend.position = "none",
        plot.title = element_text(hjust = 0.5),
        text = element_text(size = 12))

aov.MG = anovaBF( value ~ name,
                  df.pagg %>% filter(task != "CT"))

aov.MG@bayesFactor
interpret_bf(sort(aov.MG@bayesFactor$bf, decreasing = T)[1], log = T)

p2 = df.pagg %>%
  filter(task != "CT") %>%

```

```

ggplot(., aes(x = name, y = value, fill = name, colour = name)) + #
geom_rain(rain.side = 'f1x1',
          boxplot.args = list(color = "black", outlier.shape = NA, show_guide = FALSE, alpha = a),
          violin.args = list(color = "black", outlier.shape = NA, alpha = a),
          point.args = list(show_guide = FALSE, alpha = a)
        ) +
stat_summary(fun = mean, geom = "line", aes(group = 1), colour = "black") +
stat_summary(fun = mean, geom = "point", shape = 23, size = 2,
             aes(group = name)) +
stat_summary(fun = mean, geom = "point", shape = 5, size = 2,
             aes(group = name), colour = "black") +
scale_fill_manual(values = custom.col[c(1,3)]) +
scale_colour_manual(values = custom.col[c(1,3)]) +
labs(title = "Mirror game",
      x = "",
      y = "") +
theme_bw() +
theme(legend.position = "none",
      plot.title = element_text(hjust = 0.5),
      text = element_text(size = 12))

ggarrange(p1, p2, ncol = 2)

```